4.3 IOCP (I/O Completion Port)

MSDN의 IOCP 설명을 보면 IOCP는 다중처리 시스템에서 다중 비동기 IO 요청을 처리하는데 효율적인 스레딩 모델을 제공합니다. 만약 프로세스가 IOCP를 만들면 시스템은 입출력 관련된 요청을 처리하기 위해서 요청과 관련된 큐(Queue)객체를 생성합니다.
프로세스의 비동기 IO 요청이 있으면 스레드 풀에 미리 생성해 놓은 스레드와 IOCP를 가지고 작업을 처리하기 때문에 다른 어떤 처리 방식 보다 빠르고 효율적이라 합니다.

간단히 IOCP는 함수 중심의 완료 루틴을 좀 더 발전시킨 형태로 입출력(I/O) 완료 포트(Completion Port) 객체가 입출력을 처리하고 (완료)결과를 이를 통보하는 시스템입니다.
IOCP의 장점은 완료 루틴과 비교해 보면 금세 알 수 있습니다. 완료 루틴은 alertable wait 상태로 진입하는 스레드에서만 완료를 반환 받을 수 있지만 IOCP는 완료 포트의 결과를 호출하는 모든 스레드(작업 스레드)에서 완료를 반환 받을 수 있습니다. 따라서 스레드의 제약이 없고, 스레드 또한 미리 만들어 놓고 쉬고 있는 스레드를 사용할 수도 있는 구조를 가지고 있습니다.

이렇게 윈도우 시스템에서 다중 입출력 요청에 대해서 가장 효율인 IOCP는 수 백 명 이상 처리해야 하는 네트워크 프로그램에 적용한다면 좀 더 성능이 좋은 서버를 만들 수가 있게 됩니다.
IOCP 프로그램을 위해서 필요한 함수들을 살펴보겠습니다.

```
HANDLE CreateIoCompletionPort(HANDLE FileHandle
                    , HANDLE ExistingCompletionPort
                    , ULONG_PTR CompletionKey, DWORD NumberOfConcurrentThreads)
```

이 함수는 두 가지의 기능이 있습니다. 첫 번째는 IOCP 객체를 생성합니다.

```
HADLE hIocp = CreateIoCompletionPort(INVALID_HANDLE_VALUE, nullptr, 0, 0);
```

CreateIoCompletionPort() 함수의 두 번째는 역할은 IOCP 객체와 입출력 대상이 되는 객체(File Handle), 그리고 사용자가 지정한 완료 키(Completion Key)를 바인딩 하는 함수 입니다.
완료 키는 입출력 대상이 되는 객체에 대한 모든 입출력 완료 요청을 해석하는 열쇠입니다. IOCP는 완료를 끝내고 나서 함수로 완료 작업에 대한 정보를 되돌려 주는데 이 정보에는 입출력 대상이 없고 대신 사용자가 지정한 완료 키를 반환해줍니다.

```
BOOL PostQueuedCompletionStatus(HANDLE CompletionPort
                    , DWORD dwNumberOfBytesTransferred
                    , ULONG_PTR dwCompletionKey, LPOVERLAPPED lpOverlapped)
```

작업 스레드는 입출력의 결과를 통지 받는 함수입니다. 이 스레드에 어떤 특별한 내용을 전달할

필요도 있을 수 있습니다. 이러한 경우 PostQueuedCompletionStatus() 함수를 사용합니다. 이 함수는 마치 WIN API의 PostMessage()가 메시지 큐에 정보를 전달하는 것처럼 작업 스레드의 큐 영역에 메시지를 전달합니다.

```
BOOL GetQueuedCompletionStatus(HANDLE CompletionPort
                        , PDWORD lpNumberOfBytes     , PULONG_PTR lpCompletionKey
                        , LPOVERLAPPED *lpOverlapped, DWORD dwMilliseconds)
```

GetQueuedCompletionStatus(이하 GQCS) 가장 중요한 함수로 IOCP가 처리한 입출력의 결과를 통보받는 함수입니다. 만약 어떤 스레드가 이 함수를 포함하고 있다면 이 스레드를 작업(자) 스레드라 합니다.

함수의 첫 번째 인수는 입출력을 처리하는 IOCP 객체를 지정합니다. 2 ~ 4 인수는 IOCP 객체가 작업을 처리하고 정보를 반환 하는 인수입니다. 5 번째 인수는 대기 시간으로 보통 INFINITE 값으로 설정합니다.
두 번째 인수 lpNumberOfBytes는 GQCS 가 처리한 byte 수 입니다. 만약 이 값이 0으로 되어 있다면 recv() 함수의 0을 반환 받은 것과 같이 상대방이 접속을 종료한 것입니다.
세 번째 인수 lpCompletionKey는 CreateIoCompletionPort() 함수에서 ICOCP와 연결할 때 FileHandle에 대한 사용자 설정한 완료 키의 주소가 저장됩니다.
네 번째 인수 lpOverlapped는 WSASend, WSARecv 함수에서 사용한 OVERLAPPED 변수의 주소가 저장됩니다.
간단히 인수에 대해서 정리한다면 세 번째 인수는 IOCP에 작업을 요청한 객체를, 네 번째 인수는 송신 또는 수신 작업을 구분해주는 타입이라고 생각해도 됩니다.

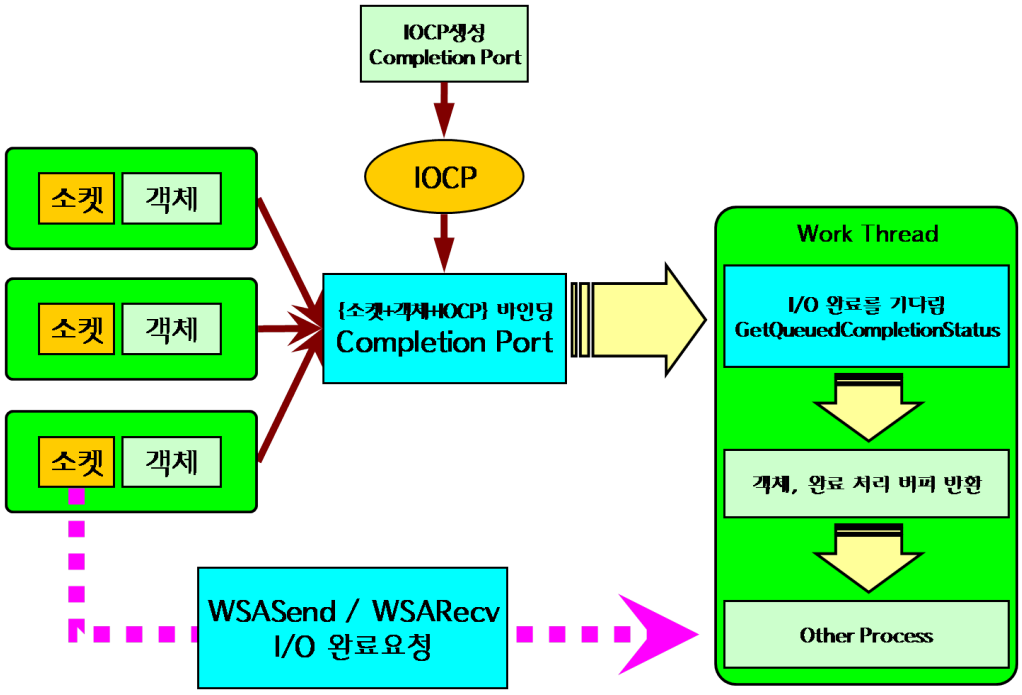
GQCS 함수에 대해서 좀더 살펴 본다면 완료 포트(completion port)가 요청한 입출력을 에러 없이 성공했다면 OGQCS 함수는 주어진 인수 lpNumberOfBytes에는 IOCP가 처리한 바이트 수, lpCompletionKey에는 IOCP와 바인딩 된 객체 주소, 그리고 lpOverlapped에는 입출력 작업 타입을 저장하고 TRUE를 반환합니다.

실패 하면 OGQCS 함수는 FALSE를 반환 합니다. 실패는 (*lpOverlapped)의 값이 nullptr 이거나 nullptr이 아닌 2 가지 유형이 있습니다. 만약 (*lpOverlapped)의 값이 nullptr로 설정되어 있다면 이것은 IOCP가 작업 자체를 완료 하지 못해 lpNumberOfBytes, lpCompletionKey, lpOverlapped에 정보도 기록되지 않는 상황입니다. 두 번째 nullptr이 아닌 경우 실패한 입출력 처리의 완료입니다. 이 때는 lpNumberOfBytes, lpCompletionKey, lpOverlapped의 인수에 값이 저장됩니다.

IOCP를 가지고 몇 가지 실험을 해보면 (*lpOverlapped)가 nullptr인 경우는 잘못된 또는 유효하지 않은 인수를 GQCS에 적용했거나 대기 시간의 초과로 인해서 발생할 수 있습니다. 예를 들어 프로

그램 중간에 GQCS 함수에 요청한 IOCP 객체가 해제되면 GQCS 함수는 실패를 반환하고 (*lpOverlapped)을 nullptr로 설정합니다. 이것을 GetLastError() 함수로 에러 코드를 확인해 보면 ERROR_ABANDONED_WAIT_0 에러 코드를 얻을 수 있습니다.
어찌 되었건 실패에 대해서 무조건 GetLastError()로 에러 코드를 얻고 처리하도록 합니다.

지금까지 IOCP에서 사용되는 함수들을 살펴보았습니다. 간단히 전체 프로그램 구조를 정리해 보면 서버는 먼저 CreateIoCompletionPort() 함수로 IOCP를 생성 합니다. 다음으로 GQCS 함수를 호출하는 작업 스레드를 2 * 프로세서 개수만큼 만들고 실행시킵니다.
만약 accept 함수에서 새로운 클라이언트의 접속을 받으면 소켓을 포함한 클라이언트 객체를 생성하고 이 객체를 완료 키로 설정합니다. <소켓, IOCP, 완료 키>를 CreateIoCompletionIoPort 함수로 바인딩 하고 클라이언트의 수신을 받기 위해 WSARecv 함수를 호출합니다.



<IOCP를 사용한 프로그램 구조>

클라이언트는 서버와 접속이 완료되고 나서 작업 스레드를 만들고, 소켓을 완료 키로 설정한 후에 <소켓, IOCP, 소켓>을 CreateIoCompletionIoPort 함수로 바인딩 합니다. 바인딩 후에 WSARecv 함수로 서버의 수신을 기다립니다.

이전의 완료 루틴을 사용한 파일 입출력 예제와 마찬가지로 IOCP 프로그램 구조를 쉽게 이해하기 위해서 파일 입출력을 연습해 보겠습니다.

먼저 다음과 같이 작업 스레드의 숫자를 정하기 위해서 시스템의 Processor 개수를 얻는 함수를

다음과 같이 GetSystemInfo() 함수를 사용해서 작성합니다.

```
int GetSystemProcessorCount()
{
        SYSTEM_INFO SystemInfo;
        GetSystemInfo(&SystemInfo);
        return (int)SystemInfo.dwNumberOfProcessors;
}
```

파일을 읽기 위해서 IOCP를 가지고 다음과 같은 순서대로 프로그램을 작성합니다.

파일 핸들과 IOCP 핸들을 선언하고 메인 함수는 이들을 생성합니다.
다음으로 작업 스레드를 ' 2 * 프로세서 개수'만큼 생성합니다.
파일 핸들을 완료 키로 설정하고 바인딩 합니다.
바인딩이 끝나면 ReadFile() 함수로 비동기 파일 읽기를 요청 합니다.

```
HANDLE  g_hFile = nullptr;      // 파일 핸들
HANDLE  g_hIocp = nullptr;      // IOCP 핸들
…
int main()
…
g_hFile = CreateFile(…); // 파일 객체 생성
g_hIocp = CreateIoCompletionPort(INVALID_HANDLE_VALUE, nullptr, 0, 0); // IOCP 객체생성
…
int nWrkPrc = 2 * GetSystemProcessorCount();
for(int i=0; i<nWrkPrc; ++i)
        HANDLE hThWrk = (HANDLE)_beginthreadex(… WorkThread, …);
…
ULONG_PTR pIoKey = (ULONG_PTR)&g_hFile;                 // IO Key is File
CreateIoCompletionPort(g_hFile, g_hIocp, pIoKey, 0);    // <IOCP, File, key> 바이딩
…
hr = AsyncRead();                                       // 비동기 Read 요청
```

작업 스레드는 GQCS 함수로 완료를 기다립니다. GQCS 함수의 반환 값에 따라 FALSE 이면 스레드를 빠져 나갑니다.
반환이 TRUE이면서 전송된 데이터가 없으면(0 == dTran) 파일을 다 읽어온 상태이므로 스레드를 빠져나갑니다.
나머지는 IOCP가 파일을 읽어온 내용을 버퍼에 저장해 놓은 상태이므로 버퍼의 내용을 화면에 출력하고 지금까지 읽어온 바이트 수를 OVERLAPPED 구조체 변수 g_rOL.Offset에 누적하고 전체 파일 길이와 비교해서 크거나 같으면 스레드를 빠져 나갑니다.
전체 파일 길이보다 작은 값이면 입력 버퍼를 초기화 하고 다시 비동기 읽기 요청을 합니다.

```
DWORD WINAPI WorkThread(void* pParam)
…
while(g_hFile)
{
```

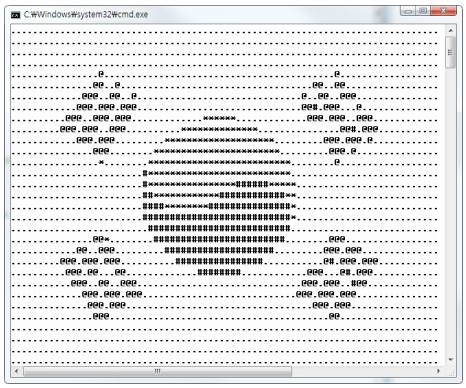
```
        ULONG_PTR       pIoKey  = nullptr;
        OVERLAPPED*     pOL     = nullptr;
        DWORD           dTran   = 0;
        DWORD           OLType  = 0;

        hr = GetQueuedCompletionStatus(
                  g_hIocp                       // Completion Port
                , &dTran                        // 전송된바이트수
                , (PULONG_PTR)&pIoKey           // 완료키 주소
                , (LPOVERLAPPED*)&pOL           // OVERLAPPED 구조체 변수 주소
                , INFINITE);
        if(0 == hr)                             // 실패
        {
                hr = GetLastError();
                break;
        }
        if(0 == dTran)                          // 전송할 데이터 없음
        {
                printf("Read Complete.\n");
                break;
        }

        printf("%s", g_rBuf);                   // 버퍼 출력

        g_rOL.Offset += dTran;                  // offset 조정(읽어온 길이)
        if(g_TotalSize <= g_rOL.Offset)         // 전체 파일 크기 비교
                break;


        memset(g_rBuf, 0, MAX_BUF);             // 버퍼 초기화
        hr = AsyncRead();                       // 비동기 입출력 요청
        …
}
…
```



<IOCP 파일 입출력: nw61_iocp_file.zip>

파일 입출력을 IOCP로 연습해보았습니다. 중첩 입출력에서 완료 통지를 이벤트를 사용했을 때와 비슷하게 서버는 수신만 IOCP로 처리하고 송신은 BSD send() 함수를 사용하겠습니다. 서버에서 클라이언트 객체를 저장할 구조체를 OVERLAPPED, WSBUF, 그리고 전송 버퍼를 포함해서 다음과 같이 선언하고 STL 벡터를 사용해서 클라이언트 리스트를 생성 합니다

```
// 클라이언트 객체용 구조체
struct RemoteHost
{
        SOCKET          scH;                    // 소켓
        OVERLAPPED      olRcv;                  // For WSARecv
        WSABUF          wsBuf;
        char            csBuf[MAX_BUF+4];       // Io Completion Buffer
        …
        void Reset()
        {
                memset(&olRcv, 0, sizeof(OVERLAPPED));
                memset(csBuf, 0, MAX_BUF);
                wsBuf.len = MAX_BUF;
                wsBuf.buf = csBuf;
        }
…
};

vector<RemoteHost* >    g_vHost;                // Client list
HANDLE  g_hIocp         = nullptr;                      // IOCP Handle
…
```

서버의 주-스레드는 CreateIoCompletionPort() IOCP 객체를 생성하고 "2 * 프로세서 개수"만큼 작업 스레드를 생성합니다.
클라이언트가 접속을 하면 클라이언트 객체를 생성하고 이 객체를 완료 키(key)로 설정한 후에 CreateIoCompletionPort() 함수를 사용해서 <클라이언트소켓, IOCP, key>를 바인딩 합니다.
접속한 클라이언트에게서 보낸 데이터를 수신할 수 있도록 WSARecv() 함수를 호출합니다.
WSARecv() 함수의 반환이 SOCKET_ERROR 이면 반드시 WSAGetLastError() 함수를 사용해서 에러 코드를 확인합니다.
WSARecv() 함수 호출 이 성공했다면 vector 컨테이너에 클라이언트 객체를 추가합니다.

```
// 서버 주-스레드
int main()
…
// IOCP 객체생성
g_hIocp = CreateIoCompletionPort(INVALID_HANDLE_VALUE, nullptr, 0, 0);

// Work 쓰레드 생성
for(int i=0; i<nWrkPrc; ++i)
        hThWrk = (HANDLE)_beginthreadex(…)
…
while(…)
{
        …
        scCln = accept(g_scLstn, …);

        RemoteHost* pCln = new RemoteHost(scCln);       // 클라이언트 인스턴스 생성
        …
        ULONG_PTR  pIoKey = (ULONG_PTR)pCln;    // Key is client Object
        …
        CreateIoCompletionPort(                 // <클라이언트 소켓, IOCP, key> binding
```

```
                (HANDLE)scCln, g_hIocp, pIoKey, 0);

        // 수신 요청
        hr = WSARecv(pCln->scH, &pCln->wsBuf, 1
                        , &dTran, &dFlag, &pCln->olRcv, nullptr);

        // 에러 반환 처리
        if(SOCKET_ERROR == hr)
        {
                hr =  WSAGetLastError();
                if(WSA_IO_PENDING != hr…)
                        …
        // add the client list
…
```

서버의 GQCS 함수를 포함하고 있는 작업 스레드는 먼저 이 함수가 반환한 성공/실패에 대해서 먼저 실패를 처리합니다.
GQCS 함수의 실패는 크게 IOCP객체, 또는 클라이언트 객체에서 발생하기 때문에 IOCP의 처리 결과를 저장하는 PULONG_PTR 타입의 완료 키 인수와 요청한 작업 종류를 파악할 수 있는 LPOVERLAPPED 인수에 어떤 정보도 기록되어 있지 않으면 IOCP객체의 문제이므로 스레드를 빠져나갑니다.
만약 클라이언트와 통신하고 있는 중에 실패하는 경우라면 완료 키를 사용해서 형 변환 등을 사용해서 클라이언트 객체를 찾고 찾은 클라이언트 객체를 해제합니다.

```
// 서버 작업 스레드
DWORD WINAPI WorkThread(void* pParam)
…
RemoteHost*     pCln    = nullptr;      // 완료 키: 클라이언트 객체 주소
OVERLAPPED*     pOlEx   = nullptr;      // WSARecv에서 사용된 OVERLAPPED 주소
DWORD           dTran   = 0;    // IOCP가 처리한 바이트 수
while(0 < g_scLstn)
{
        hr = GetQueuedCompletionStatus( g_hIocp, &dTran
                , (PULONG_PTR)&pCln, (LPOVERLAPPED*)&pOlEx, INFINITE);

        if(0 == hr)     // 실패를 반환 받을 때
        {
                …
                if(nullptr == pOlEx && nullptr == pCln) // IOCP에서 에러 발생
                        break;
                if(pOlEx && pCln)               // 클라이언트에서 에러 발생
                        …
                continue;
        }
```

GQCS 함수의 성공 결과를 처리하는 과정에서 GQCS 함수의 두 번째 인수인 IOCP가 처리한 바이트 수가 0 이 되면 클라이언트가 접속을 종료한 것이므로 클라이언트 객체를 해제합니다.

다음으로 WSARecv 함수로 수신 요청된 데이터는 WSABUF에서 지정한 버퍼 주소에 저장될 것입니다. 이 버퍼에 저장된 데이터를 가지고 나머지 일을 처리합니다.
다시 클라이언트가 보낸 데이터를 수신할 수 있도록 WSARecv() 함수를 호출합니다.

```
// 서버 작업 스레드 WorkThread 계속
…
        SOCKET  scHost  = pCln->scH;
        // 접속 클라이언트 종료 처리
        if(0 == dTran)
        {
                printf("Disconnect Client: %d\n", (int)scHost);
                …
        }
        …
        // 수신완료 처리
        printf("Recv from Client[%4d]: %s\n", (int)scHost, pCln->csBuf);
        …
        // 비동기 수신 요청
        hr = WSARecv(…);
        …
```

<IOCP 서버 수신: nw62_iocp_basic.zip>

서버에서 IOCP를 사용해서 데이터를 수신을 해봤습니다. 클라이언트는 구조가 간단하기 때문에 수신 뿐만 아니라 송신도 IOCP 사용해보겠습니다.

IOCP를 사용하기 위해서 다음과 같이 IOCP 핸들, 송수신 OVERLAPPED 구조체 변수, 송수신 버퍼 등을 선언합니다.

```
HANDLE          g_hIocp  = nullptr;             // IOCP 핸들

// 송신에 필요한 변수들
OVERLAPPED      g_sndOL   ={0};
DWORD           g_sndTran = 0;
WSABUF          g_sndWsBuf={0};
char            g_sndBuf[MAX_BUF+4]={0};        // 송신용 버퍼

// 수신에 필요한 변수들
OVERLAPPED      g_rcvOL   ={0};
DWORD           g_rcvTran = 0;
WSABUF          g_rcvWsBuf={0};
char            g_rcvBuf[MAX_BUF+4]={0};        // 수신용 버퍼
```

클라이언트에서 필요한 비동기 입출력 함수 사용은 이벤트 통지를 사용한 중첩 입출력과 완료 루틴에서 살펴보았으므로 설명을 생략하겠습니다.

```
INT AsyncSend(char* s, int l)
```

```
…
        memcpy(g_sndBuf, s, l);
        g_sndWsBuf.len = l;
        g_sndWsBuf.buf = g_sndBuf;
        int hr= WSASend(g_scHost, …);
…
        return hr;
…
INT AsyncRecv()
…
        g_rcvWsBuf.len = MAX_BUF;
        g_rcvWsBuf.buf = g_rcvBuf;
        return WSARecv(g_scHost, …);
…
```

클라이언트의 주-스레드는 접속을 성공하면 IOCP, 작업 스레드를 생성하고, 완료 키를 소켓으로 설정해서 이들을 CreateIoCompletionPort 함수로 바인딩 합니다.
서버에서 보낼 데이터를 수신하기 위해 비동기 함수로 수신을 요청합니다.

```
// 클라이언트
int main()
…
hr = connect(…);                        // 서버 접속
…
g_hIocp = CreateIoCompletionPort(…);    // IOCP 객체생성
…
_beginthreadex(…);                      // Work 쓰레드 생성
…
ULONG_PTR pIoKey = (ULONG_PTR)&g_scHost; // 소켓을 완료 키로 설정

// <소켓, IOCP, key> 바이딩
CreateIoCompletionPort((HANDLE)g_scHost, g_hIocp, pIoKey, 0);
…
AsyncRecv();                            // 비동기수신요청
…
```

작업 스레드에서 실패와 성공에서 전송된 바이트 수가 0일 경우는 IOCP 객체가 문제이거나 서버와 접속이 해제된 상황이므로 스레드를 빠져 나갑니다.
성공에서 GQCS 함수에서 얻은 LPOVERLAPPED 인수와 송신 또는 수신 OVERLAPPED 구조체 변수의 주소와 비교에서 송신, 수신 완료를 처리합니다.
마지막으로 수신 완료 처리에서 수신을 요청해서 이후에 서버에서 보낸 데이터를 수신할 수 있도록 합니다.

```
// 클라이언트 작업 스레드
DWORD WINAPI WorkThread(void* pParam)
…
while(…)
{
        hr = GetQueuedCompletionStatus( g_hIocp , &dTran
```

```
                , (PULONG_PTR)&pIoKey,  (LPOVERLAPPED*)&pOL, INFINITE);

        if(0 == hr)
                …
                break;

        if(0 == dTran)                          // 접속 종료
                …
                break;

        // 송신 완료
        if( ((void*)&g_sndOL) == ((void*)pOL) )
        {
                …
                printf("Write Successed.\n");
        }
        // 수신 완료
        else if( ((void*)&g_rcvOL) == ((void*)pOL) )
        {
                …
                printf("Recv from Server(%3d): %s\n", dTran, g_rcvBuf);
                AsyncRecv();                    // 수신 요청
        }
}
```

<IOCP: nw62_iocp_basic.zip>

지금까지 IOCP 를 사용해서 서버에서는 수신을 클라이언트는 송수신을 해보았습니다. 다음으로 채팅 프로그램을 만들어서 서버의 송수신을 완벽하게 IOCP를 사용하고 클라이언트도 서버에서 사용한 구조체로 프로그램을 구성해 보겠습니다.
완료 루틴에서 사용했던 것처럼 WSAOVERLAPPED 구조체를 확장해서 송신과 수신을 처리할 수 있도록 확장 구조체 OVERLAP_EX를 구성합니다.

```
struct OVERLAP_EX : public WSAOVERLAPPED
{
        DWORD   dType;                  // IO 타입. FD_READ 또는 FD_WRITE
        DWORD   dTran;                  // 전송된 바이트 수
        DWORD   dFlag;                  // Flag = 0
        WSABUF  wsBuf;                  // WSABUF
        char    csBuf[MAX_BUF+4];       // 송신 또는 수신 버퍼
…
        void SetBuf(char* s, int l=0)   // 버퍼에 데이터 설정
        {
                if(s && 0<l)
                {
                        wsBuf.len = l;
                        memcpy(csBuf, s, l);
                }
                else
                {
                        wsBuf.len = MAX_BUF;
```

```
                }
        }

        int AsyncRcv(SOCKET s)          // 비동기 수신
        {
                return WSARecv(s, &wsBuf, 1, &dTran, &dFlag, this, nullptr);
        }

        int AsyncSnd(SOCKET s)          // 비동기 송신
        {
                int hr = WSASend(s, &wsBuf, 1, &dTran, dFlag, this, nullptr);
                if(0 == hr)             // 성공하면 작업 큐에 저장 안됨
                        printf("Send complete: %d byte\n", dTran);

                return hr;
        }
};
```

클라이언트 객체에 대한 구조체에서 완료 루틴 때와 유사하게 송신용, 수신용 확장 구조체 필드를 구성합니다. 그리고 소켓을 설정하는 생성자 함수에서 확장 구조체의 작업을 FD_XXX 값으로 설정합니다.
멤버 함수로 AsyncSend, AsyncRecv 함수를 포함합니다.

```
// 접속자용 구조체
struct RemoteHost
{
        OVERLAP_EX      olSnd;  // WSASend 함수용
        OVERLAP_EX      olRcv;  // WSARecv 함수용
…
        RemoteHost(SOCKET s)
        {
                …
                olSnd.dType     = FD_WRITE;
                olRcv.dType     = FD_READ;
        }
…
        INT     AsyncSend(char* s, int l)
        {
                olSnd.Reset();
                olSnd.SetBuf(s, l);
                return olSnd.AsyncSnd(scH);
        }
        INT     AsyncRecv()
        {
                olRcv.Reset();
                return olRcv.AsyncRcv(scH);
        }
};
…
vector<RemoteHost* >    g_vHost;        // Client 리스트
HANDLE  g_hIocp         = nullptr;              // IOCP 핸들
```

확장 구조체의 'dType' 이라는 멤버 변수로 송신 요청인지 수신 요청인지 판별할 수가 있습니다. 이전의 서버 코드에 이 변수를 비교해서 작업을 분류하고 입출력에 대한 나머지 과정을 처리합니다. 특히, 수신 요청에 대한 작업이라면 마지막 단계에서 비동기 수신을 다시 요청합니다.

```
// 서버 작업 스레드
DWORD WINAPI WorkThread(void* pParam)
…
while(…)
{
        hr = GetQueuedCompletionStatus(…);

        // IO Failed
        …
        // 접속 종료(0 == dTran)
        …
        // 송신완료
        if(FD_WRITE == pOlEx->dType)
        {
                // 송신완료 처리
                …
        }
        // 수신완료
        else if(FD_READ == pOlEx->dType)
        {
                …
                // 비동기 수신 요청
                pCln->AsyncRecv();
        }
…
```

<IOCP 채팅 서버: nw63_iocp_chat.zip>

클라이언트는 서버에서 사용한 확장 구조체를 그대로 이용합니다. 작업 스레드도 확장 구조체 변수를 이용해서 송신 또는 수신 요청 작업을 분리하고 이에 대한 나머지 작업을 수행합니다. 수신 요청이면 마지막에 비동기 수신을 요청합니다.

비동기 입출력은 송수신 처리 이외에 나머지 connection, accept 를 처리하기가 어렵습니다. 만약 이들을 작업스레드에서 처리하고자 한다면 작업 스레드에게 메지지를 전달해야 합니다. 이전에 소개한 PostQueuedCompletionStatus () 함수는 이런 상황에 대처하는데 알맞은 함수입니다.

다음 클라이언트 예제는 접속을 성공하고 나서 작업 스레드에게 접속을 알리고 있습니다. Post-QueuedCompletionStatus() 함수의 인수로 사용하는 키에 FD_ACCEPT를 설정하고 작업 스레드에서 키의 값을 FD_ACCEPT로 비교하고 있습니다.
이런 방법은 서버에서도 사용될 수 있으며 작업 스레드에게 뭔가를 요청하고 싶을 때 주저하지 말고 PostQueuedCompletionStatus () 함수를 사용하기 바랍니다.

```
// 채팅 클라이언트
SOCKET          g_scHost = 0;                   // listen socket
OVERLAP_EX      g_olSnd(FD_WRITE);
OVERLAP_EX      g_olRcv(FD_READ);
HANDLE          g_hIocp  = nullptr;             // IOCP Handle

INT     AsyncSend(char* s, int l)               // 데이터 송신
{
        g_olSnd.Reset();
        g_olSnd.SetBuf(s, l);
        return g_olSnd.AsyncSnd(g_scHost);
}

INT     AsyncRecv()                             // 데이터 수신
{
        g_olRcv.Reset();
        return g_olRcv.AsyncRcv(g_scHost);
}

…
int main()
{
        hr = connect(…);

        ULONG_PTR pIoKey = (ULONG_PTR)&g_scHost; // 완료키 = 소켓
        CreateIoCompletionPort(…);              // <소켓, IOCP, key> 바이딩

        // 작업 스레드에게 접속 알림을 설정
        ULONG_PTR       dKey = FD_CONNECT;      // 키를 FD_CONNECT로 설정
        WSAOVERLAPPED   tOl={0};                // dummy
        hr = PostQueuedCompletionStatus(g_hIocp, sizeof(ULONG_PTR), dKey, &tOl);

        // 비동기수신요청
        AsyncRecv();
…


DWORD WINAPI WorkThread(void* pParam)
…
while(g_scHost)
{
        hr = GetQueuedCompletionStatus(…);

        // IO Failed
        …
        // 접속 종료
        …
        // PostQueuedCompletionStatus() 함수로 보낸 접속 메시지
        if(sizeof(ULONG_PTR) == dTran && FD_CONNECT == pIoKey)
        {
                printf("Connection Successed\n");
                continue;
        }

        OLType= pOL->dType;                     // 타입
        if(FD_READ == OLType)                   // receiving complete
```

```
        {
                // 수신 작업 처리
                …
                AsyncRecv();                    // 비동기수신요청
        }
        else if(FD_WRITE == OLType)             // Sending complete
        {
                // 송신 작업 처리
        }
…
```

<IOCP: nw63_iocp_chat.zip>